

ใบความรู้ที่ 7 โครงข่ายประสาทเทียมพื้นฐาน (Basic Neural Networks)

วัตถุประสงค์การเรียนรู้

1. รู้ความหมายและที่มาของแนวคิดโครงข่ายประสาทเทียม (Neural Networks) ที่เลียนแบบการทำงานของสมองมนุษย์
2. เข้าใจองค์ประกอบพื้นฐานของ Perceptron และบทบาทของฟังก์ชันกระตุ้น (Activation Function) เช่น ReLU ในการสร้างความฉลาดให้โมเดล
3. เข้าใจหลักการทำงานของ Convolutional Neural Networks (CNN) และกระบวนการ Convolutions ที่ใช้ในการประมวลผลและจำแนกรูปภาพ
4. รู้จักหน้าที่ของเลเยอร์ต่างๆ ในสถาปัตยกรรม CNN เช่น Conv2D, MaxPooling2D, Flatten และ Dense
5. เข้าใจกระบวนการสร้างโมเดลสำหรับจำแนกตัวเลขเขียนด้วยมือ (MNIST Classification) ตั้งแต่การเตรียมข้อมูลจนถึงการวัดผล
6. เข้าใจหลักการทำงานของระบบจดจำใบหน้า (Face Recognition) ที่ใช้เทคนิค Transfer Learning ร่วมกับสถาปัตยกรรม InceptionResnetV1
7. รู้วิธีการวัดความเหมือนของข้อมูลใบหน้าด้วยหลักการ Cosine Similarity และ Metric Learning

1. รากฐานทางทฤษฎี: จากชีววิทยาสู่คณิตศาสตร์ (Theoretical Foundations: From Biology to Mathematics)

1.1. แรงบันดาลใจจากธรรมชาติ (Biological Inspiration)

ความพยายามของมนุษย์ในการสร้างเครื่องจักรที่ "คิด" ได้ มิได้เริ่มต้นที่ห้องปฏิบัติการคอมพิวเตอร์ แต่เริ่มต้นจากการศึกษาเครื่องจักรที่คิดได้ดีที่สุดและซับซ้อนที่สุดในจักรวาล นั่นคือ "สมองมนุษย์" สมองของเราประกอบด้วยเซลล์ประสาทหรือ Neuron (นิวรอน) จำนวนมหาศาลประมาณ 86,000 ล้านเซลล์ ซึ่งเชื่อมต่อกันเป็นเครือข่ายร่างแหที่ซับซ้อนและทรงพลัง การทำงานของ Neuron ทางชีววิทยาสามารถแบ่งออกเป็น 3 ขั้นตอนหลัก ซึ่งได้กลายเป็นต้นแบบเชิงโครงสร้าง (Structural Archetype) ของปัญญาประดิษฐ์

1.1.1. การรับสัญญาณ (Reception - Dendrites): เซลล์ประสาทจะมีเส้นใยที่แตกแขนง คล้ายกิ่งไม้จำนวนมากเรียกว่า Dendrite (เดนไดรต์) ทำหน้าที่รับสัญญาณไฟฟ้าเคมีจากเซลล์ประสาทข้างเคียง เปรียบเสมือน "ช่องทางนำเข้าข้อมูล" (Input Channels) ของระบบ

1.1.2. การประมวลผล (Processing - Soma): สัญญาณทั้งหมดที่ได้รับจะถูกรวบรวมและส่งไปยังตัวเซลล์ (Soma หรือ Cell Body) ภายในนิวเคลียสจะเกิดกระบวนการรวมสัญญาณ (Summation) หากผลรวมของสัญญาณกระตุ้นมีความเข้มข้นเกิน "ค่าขีดเริ่มเปลี่ยน" (Threshold) เซลล์จะถูกกระตุ้นให้ทำงาน (Fire) แต่ถ้าสัญญาณอ่อนเกินไป เซลล์ก็จะเงียบและไม่ส่งต่อข้อมูล

1.1.3. การส่งต่อสัญญาณ (Transmission - Axon & Synapse): เมื่อเซลล์ถูกกระตุ้น มันจะส่งกระแสประสาท (Action Potential) วิ่งผ่านเส้นใยยาวเดี่ยวที่เรียกว่า Axon (แอกซอน) ไปยังปลายประสาท เพื่อส่งต่อไปยัง Dendrite ของเซลล์อื่นผ่านจุดเชื่อมต่อเล็กๆ ที่เรียกว่า Synapse (ไซแนปส์)

1.2. เพอร์เซปตรอน (Perceptron): เซลล์ประสาทในรูปสมการคณิตศาสตร์

นักวิทยาศาสตร์คอมพิวเตอร์และนักคณิตศาสตร์ในยุคบุกเบิก เช่น Frank Rosenblatt ได้นำโมเดลทางชีววิทยานี้มาแปลงเป็นแบบจำลองทางคณิตศาสตร์ที่เรียกว่า Perceptron ซึ่งถือเป็นหน่วยย่อยที่สุด (Fundamental Unit) ของโครงข่ายประสาทเทียม ในโลกของคอมพิวเตอร์ เราไม่ได้ใช้สารเคมีหรือประจุไฟฟ้าในการสื่อสาร แต่เราใช้ "ตัวเลข" (Numerical Data) ในการแทนข้อมูล

สมการพื้นฐานของ Perceptron หนึ่งตัว มีความคล้ายคลึงอย่างน่าทึ่งกับสมการ Linear Regression คือ:

$$z = \sum_{i=1}^n w_i x_i + b$$

หรือเขียนในรูปแบบเวกเตอร์ได้ว่า:

$$z = w \cdot x + b$$

โดยที่องค์ประกอบแต่ละส่วนมีความหมายเชิงเปรียบเทียบดังนี้:

1.2.1. x (Input) : คือ ข้อมูลขาเข้า (เปรียบเสมือนสัญญาณที่ Dendrite รับมา) ในบริบทของคอมพิวเตอร์วิทัศน์ ค่านี้อาจเป็นความเข้มของแสงในแต่ละพิกเซลของรูปภาพ หรือข้อมูลตัวเลขเชิงปริมาณอื่นๆ

1.2.2. **w** (Weight): คือ ค่าน้ำหนัก (เปรียบเสมือนความแข็งแรงของ Synapse) เป็นตัวแปรที่สำคัญที่สุดที่ AI ต้อง "เรียนรู้" (Learnable Parameter) ค่าน้ำหนักจะเป็นตัวกำหนดว่า Input ตัวไหนมีความสำคัญมากหรือน้อยต่อผลลัพธ์สุดท้าย ถ้า **w** มีค่ามาก แสดงว่าข้อมูลช่องทางนั้นมีอิทธิพลสูง หาก **w** เป็นลบ แสดงว่าข้อมูลนั้นมีผลลบ

1.2.3. **b** (Bias): คือ ค่าไบแอส หรือจุดตัดแกน (Intercept) ช่วยปรับสมดุลของสมการให้ยืดหยุ่นขึ้น เปรียบเสมือนระดับพื้นฐานในการกระตุ้นเซลล์ (Activation Threshold) ที่ทำให้โมเดลสามารถเลื่อนกราฟขึ้นลงได้โดยไม่ต้องอิงกับจุดกำเนิด (0,0) เพียงอย่างเดียว [1, 1]

1.2.4. **z** (Weighted Sum): คือ ผลรวมเชิงเส้นของข้อมูลและน้ำหนัก ซึ่งเป็นผลลัพธ์ดิบ (Logits) ก่อนเข้าสู่กระบวนการตัดสินใจ

1.3. พลังแห่งความไม่เป็นเส้นตรง: ฟังก์ชันกระตุ้น (Activation Function)

หากโครงข่ายประสาทเทียมมีเพียงสมการ $z = wx + b$ เพียงอย่างเดียว ต่อให้เรานำ Perceptron มาเชื่อมต่อกันเป็นล้านตัว (Stacking Layers) ผลลัพธ์ทางคณิตศาสตร์ของมันก็จะยังคงเป็นเพียง "สมการเส้นตรง" (Linear Equation) ที่ซับซ้อนขึ้นเท่านั้น ซึ่งในทางทฤษฎีแล้ว มันจะไม่สามารถแก้โจทย์ที่มีความซับซ้อนสูงได้ เช่น การแยกแยะหน้าคน การแปลภาษา หรือการขับรถอัตโนมัติ ซึ่งล้วนเป็นโจทย์แบบ Non-linear Problem (ปัญหาที่ไม่เป็นเส้นตรง) จุดนี้เองที่ Activation Function (ฟังก์ชันกระตุ้น) เข้ามามีบทบาทสำคัญ เปรียบเสมือน "สวิตช์อัจฉริยะ" ที่ทำหน้าที่ตัดสินใจว่าเซลล์ประสาทรุ่นนั้นควรจะ "ยิง" (Fire) สัญญาณออกไปหรือไม่ และยิ่งด้วยความแรงเท่าใด การใส่ Activation Function เข้าไปที่ปลายทางของสมการ ($y = \sigma(z)$) ทำให้โครงข่ายมีความสามารถในการเรียนรู้ความสัมพันธ์ที่โค้งงอ บิดเบี้ยว และซับซ้อนของข้อมูลในโลกความเป็นจริงได้

1.3.1. เจาะลึก ReLU: ความเรียบง่ายที่ทรงพลัง

ReLU มีหลักการว่า "ถ้าค่าติดลบ ให้ตัดเป็น 0 แต่ถ้าเป็นบวก ให้ปล่อยผ่าน" ความเรียบง่ายนี้กลับเป็นกุญแจสำคัญที่ทำให้ Deep Learning ประสบความสำเร็จ เพราะ:

1.3.1.1. Computational Efficiency: คอมพิวเตอร์สามารถคำนวณ ReLU ได้เร็วกว่า Sigmoid ที่ต้องยกกำลัง Exponential หลายเท่า ทำให้เราสามารถสร้างโมเดลที่มีความลึก (Depth) เป็นร้อยชั้นได้

1.3.1.2. Linearity in Positive Region: ในช่วงที่ค่าเป็นบวก ความชัน (Gradient) จะเป็น 1 เสมอ ทำให้การส่งต่อข้อผิดพลาด (Backpropagation) ไปยังเลเยอร์ต้นๆ ทำได้ดี โมเดลจึงเรียนรู้ได้เร็วและไม่หยุดเรียนรู้กลางคัน

2. สถาปัตยกรรมโครงข่ายประสาทเทียมแบบคอนโวลูชัน (Convolutional Neural Networks: CNN)

เมื่อเราขยับจากการทำนายตัวเลขข้อมูลทั่วไปมาสู่โลกของ "รูปภาพ" (Computer Vision) โครงข่ายประสาทแบบดั้งเดิม (Fully Connected Network / Dense Layer) เริ่มพบกับข้อจำกัดทางกายภาพและทางคณิตศาสตร์มหาศาล

2.1. ข้อจำกัดของโครงข่ายแบบดั้งเดิมกับงานภาพ (The Limitation of Dense Layers)

สมมติเราต้องการประมวลผลภาพถ่ายความละเอียดระดับ Full HD (1920×1080) หากเราใช้เทคนิคแบบ Fully Connected เราจะต้องแปลงภาพนี้เป็นเวกเตอร์ยาวประมาณ 2 ล้านตัวเลข ($1920 \times 1080 \approx 2 \times 10^6$) หาก Layer ถัดไปมี Neuron เพียง 1,000 ตัว จำนวนน้ำหนัก (w) ที่โมเดลต้องเรียนรู้จะเท่ากับ $2 \times 10^6 \times 10^3 = 2,000$ ล้านพารามิเตอร์!

ผลเสียที่ตามมาคือ:

2.1.1. Computational Cost: ใช้ทรัพยากรคำนวณและหน่วยความจำมหาศาลเกินความจำเป็น

2.1.2. Overfitting Risk: ด้วยจำนวนพารามิเตอร์ที่มากขนาดนี้ โมเดลจะจดจำรายละเอียดทุกอย่างของภาพฝึกสอน (รวมถึง Noise) จนไม่สามารถนำไปใช้กับภาพใหม่ได้

2.1.3. Loss of Spatial Structure: การแปลงภาพ 2 มิติให้เป็นแถวเดียว (Flattening) เพื่อเข้า Dense Layer เป็นการทำลาย ความสัมพันธ์เชิงพื้นที่ (Spatial Structure) ของภาพไปจนหมดสิ้น (เช่น พิกเซลของตาซ้ายที่ควรอยู่ข้างจมูก อาจถูกย้ายไปอยู่ไกลกันมากในเวกเตอร์) ทำให้โมเดลจับรูปแบบรูปร่าง (Shape) ได้ยาก

2.2. คอนโวลูชัน: การมองโลกผ่านหน้าต่าง (The Convolution Operation)

Convolutional Neural Network (CNN) ถูกออกแบบมาเพื่อแก้ปัญหานี้โดยเลียนแบบกระบวนการมองเห็นของสิ่งมีชีวิตใน Visual Cortex แทนที่จะมองภาพทั้งภาพพร้อมกันในคราวเดียว CNN ใช้วิธีการสแกนภาพทีละส่วนด้วย "แผ่นกรอง" ขนาดเล็กที่เรียกว่า Filter หรือ Kernel

2.2.1. กลไกทางคณิตศาสตร์ของ Convolution (Conv2D)

กระบวนการ Convolution คือการนำ Kernel (ซึ่งเป็นเมทริกซ์ขนาดเล็ก เช่น 3×3) มาวางทับบนพื้นที่ส่วนหนึ่งของรูปภาพ แล้วคำนวณผลคูณแบบจุด (Element-wise Dot Product) ระหว่างค่าใน Kernel กับค่าพิกเซลในภาพ จากนั้นนำผลคูณทั้งหมดมารวมกันเป็นค่าเดียว แล้วขยับ Kernel ไปยังตำแหน่งถัดไปเรื่อยๆ จนทั่วทั้งภาพ

2.2.2. สมการสังเขปของการคำนวณ Output (Feature Map)

สำหรับพิกเซลที่ตำแหน่ง (i, j) ใน Feature Map ผลลัพธ์ $S(i, j)$ เกิดจาก:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

โดยที่ I คือเมทริกซ์ของภาพขาเข้า และ K คือเมทริกซ์ของ Kernel

ตัวอย่างการคำนวณจริง (Numerical Example):

กำหนดให้ภาพ Input ขนาด 4×4 และ Kernel ขนาด 3×3 ดังนี้

Input (I):

0	3	6	9
1	4	7	10
2	5	8	11
3	6	9	12

Kernel (K):

0	1	0
0	1	0
0	1	0

เมื่อวาง Kernel ทาบที่มุมซ้ายบนของภาพ (ตำแหน่ง $x = 1, y = 1$) การคำนวณจะเป็นดังนี้:

$$(0 \times 0) + (3 \times 1) + (6 \times 0) + (1 \times 0) + (4 \times 1) + (7 \times 0) + (2 \times 0) + (5 \times 1) + (8 \times 0)$$

$$= 0 + 3 + 0 + 0 + 4 + 0 + 0 + 5 + 0 = 12$$

ผลลัพธ์ 12 นี้จะเป็นพิกเซลแรกของภาพใหม่ (Feature Map) กระบวนการนี้ช่วยให้โมเดลสามารถ "สกัดฟีเจอร์" (Feature Extraction) เช่น เส้นแนวตั้ง เส้นแนวนอน หรือขอบวัตถุ ออกมาได้ โดยค่าน้ำหนักใน Kernel คือสิ่งที่โมเดลจะเรียนรู้เองระหว่างการเทรน

2.3. พารามิเตอร์ควบคุมโครงสร้าง (Hyperparameters of CNN)

การออกแบบ CNN ต้องอาศัยการกำหนดค่าพารามิเตอร์สำคัญ 3 ประการ :

2.3.1. Filter Size (Kernel Size): ขนาดของหน้าต่างที่ใช้สแกน ปกตินิยมใช้ 3×3 หรือ 5×5

2.3.1.1. ขนาดเล็ก (3×3): จับรายละเอียดที่ละเอียดอ่อน (Fine details)

2.3.1.2. ขนาดใหญ่ (5×5 หรือ 7×7): จับโครงสร้างภาพรวม (Global features) แต่มักใช้เฉพาะเลเยอร์แรก ๆ

2.3.2. Stride (การก้าวกระโดด): ระยะที่ Kernel ขยับในแต่ละครั้ง

2.3.2.1. Stride = 1: ขยับทีละ 1 พิกเซล (ละเอียดสุด ได้ภาพผลลัพธ์ขนาดใหญ่เต็ม)

2.3.2.2. Stride = 2: ขยับทีละ 2 พิกเซล (ข้ามพิกเซล) ทำให้ขนาดภาพผลลัพธ์ลดลงครึ่งหนึ่ง เป็นการลดขนาดข้อมูล (Downsampling) ไปในตัว

2.3.3. Padding (การเติมขอบ): การเติมพิกเซล (มักเป็นเลข 0) รอบๆ ภาพต้นฉบับก่อนทำ Convolution

2.3.3.1. Valid Padding: ไม่เติมขอบ ภาพผลลัพธ์จะเล็กลงเรื่อยๆ (ขอบหาย)

2.3.3.2. Same Padding: เติมขอบเพื่อให้ภาพผลลัพธ์มีขนาดเท่ากับภาพขาเข้า เพื่อรักษาข้อมูลบริเวณขอบภาพไว้

2.3.4. สูตรคำนวณขนาดภาพหลัง Convolution:

$$Output_Size = \frac{Input_Size - Kernel_Size + 2 \times Padding}{Stride} + 1$$

2.4. การลดรูปข้อมูล: Pooling Layer

หลังจากผ่าน Convolution และ Activation Function (ReLU) เราจะได้ Feature Map ที่บอกตำแหน่งของฟีเจอร์ต่างๆ ขั้นตอนต่อไปคือการลดขนาดข้อมูล (Dimensionality Reduction) เพื่อให้ประมวลผลเร็วขึ้น ลดจำนวนพารามิเตอร์ และทำให้โมเดลทนทานต่อการขยับเขยื้อนหรือบิดเบี้ยวเล็กน้อยของวัตถุ (Translation Invariance) กระบวนการนี้เรียกว่า Pooling

2.4.1. Max Pooling: เลือกค่าที่ "มากที่สุด" ในหน้าต่างนั้น (เช่น ขนาด 2×2) เป็นตัวแทนของพื้นที่ วิธีนี้เป็นที่นิยมที่สุดในงานจำแนกภาพ เพราะเป็นการดึง "ฟีเจอร์ที่เด่นชัดที่สุด" (Most Activated Feature) ออกมา เช่น ถ้าเจอลายเส้นที่มุมไหน ก็จะเก็บค่านั้นไว้

2.4.2. Average Pooling: หาค่าเฉลี่ยของพื้นที่ วิธีนี้จะให้ภาพรวมที่นุ่มนวลกว่า (Smooth) แต่มักจะทำให้ความคมชัดของฟีเจอร์ลดลง ปัจจุบันนิยมใช้เฉพาะในขั้นตอนสุดท้ายก่อนเข้าสู่การจำแนกผล (Global Average Pooling)

3. กรณีศึกษาที่ 1: การจำแนกตัวเลขเขียนด้วยมือ (Case Study: MNIST Classification)

เพื่อให้ผู้เรียนสามารถนำทฤษฎีไปสู่การปฏิบัติได้จริง (Active Learning) เราจะศึกษาผ่านโปรเจกต์ "Hello World" ของวงการ Computer Vision นั่นคือการสร้าง AI สำหรับจำแนกตัวเลข 0-9 จากชุดข้อมูล MNIST

3.1. ความสำคัญทางประวัติศาสตร์ของ MNIST

ชุดข้อมูล MNIST (Modified National Institute of Standards and Technology) ประกอบด้วยภาพตัวเลขเขียนด้วยมือขาว-ดำ (Grayscale) ขนาด 28×28 พิกเซล จำนวนรวม 70,000 ภาพ (60,000 สำหรับเทรน และ 10,000 สำหรับทดสอบ) ข้อมูลนี้ถูกสร้างขึ้นโดย Yann LeCun และคณะ เพื่อใช้เป็นมาตรฐาน (Benchmark) ในการเปรียบเทียบประสิทธิภาพของอัลกอริทึม AI ต่างๆ ความเรียบง่ายแต่มีความท้าทายในตัวทำให้ MNIST เป็นจุดเริ่มต้นที่ดีที่สุดสำหรับการเรียนรู้เรื่อง CNN ก่อนจะไปสู่โจทย์ที่ซับซ้อนกว่า

3.2. วิศวกรรมข้อมูลและการเตรียมพร้อม (Data Engineering & Preprocessing)

3.2.1. Normalization (การปรับบรรทัดฐาน): ข้อมูลภาพดิจิทัลมีค่าความสว่างของพิกเซลในช่วง 0-255 ซึ่งเป็นช่วงที่กว้างเกินไปสำหรับการคำนวณใน Neural Network ทำให้การปรับค่าน้ำหนัก (Weight Update) ทำได้ช้าและไม่เสถียร การหารค่าพิกเซลด้วย 255.0 จะช่วยบีบข้อมูลให้อยู่ในช่วง 0.0 - 1.0 (Floating point) ซึ่งช่วยให้โมเดลลู่เข้าสู่จุดต่ำสุด (Converge) ได้เร็วขึ้น

3.2.2. Reshaping (การปรับมิติ): ภาพ 28×28 แบบดั้งเดิมใน Dataset มักถูกเก็บในรูปแบบ 2 มิติ แต่ Conv2D Layer ของไลบรารีส่วนใหญ่ (เช่น TensorFlow/Keras) ต้องการ Input แบบ 3 มิติ คือ (ความสูง, ความกว้าง, จำนวนช่องสี) ดังนั้นเราต้องแปลงข้อมูลให้เป็นรูปทรง $(28,28,1)$ เพื่อระบุว่าเป็นภาพขาวดำ (1 Channel)

3.2.3. One-Hot Encoding: การแปลงข้อมูลป้ายกำกับ (Label) จากตัวเลขโดด (Integer) เช่น "เลข 5" ให้เป็นเวกเตอร์ความน่าจะเป็น เช่น `` เพื่อให้สอดคล้องกับ Output ของโมเดลที่เป็นความน่าจะเป็น และใช้ในการคำนวณค่า Loss แบบ Categorical Crossentropy ได้

3.3. การวิเคราะห์สถาปัตยกรรมโมเดล (Model Architecture Analysis)

จากการถอดรหัส (Deconstruct) เราสามารถสร้างแผนภาพสถาปัตยกรรมของ CNN ที่ใช้ได้ดังตารางต่อไปนี้:

ลำดับ (Layer)	ประเภท (Type)	พารามิเตอร์ (Parameters)	หน้าที่และหลักการทำงาน (Function & Mechanism)
Input	Image Input	Shape: (28, 28, 1)	รับข้อมูลภาพที่ผ่านการ Normalize แล้ว
1	Conv2D	32 filters, 3x3 kernel, ReLU	Feature Extraction: ทำหน้าที่เป็น "ดวงตา" ด้านแรก สกัดฟีเจอร์พื้นฐานระดับต่ำ (Low-level features) เช่น เส้นขอบ แนวตั้ง, แนวนอน, มุมเฉียง จากภาพต้นฉบับ
2	MaxPooling2D	2x2 pool size	Downsampling: ลดขนาดภาพลง 50% (เหลือ 14x14) โดยคัดเลือกเฉพาะจุดที่ฟีเจอร์เด่นชัดที่สุด เพื่อลดภาระการคำนวณ
3	Conv2D	64 filters, 3x3 kernel, ReLU	Deep Feature Extraction: สกัดฟีเจอร์ระดับสูงขึ้นไป (High-level features) จากผลลัพธ์ขั้นก่อนหน้า เช่น เริ่มมองเห็นส่วนโค้งของเลข 8 หรือหัวของเลข 9
4	MaxPooling2D	2x2 pool size	ลดขนาดภาพอีกครั้ง (เหลือ 7x7) เพื่อบีบอัดข้อมูลสำคัญให้เข้มข้นขึ้น
5	Flatten	-	Transformation: เปลี่ยนรูปแบบข้อมูลจาก "ภาพ 2 มิติ" (Feature Map) ให้กลายเป็น "เวกเตอร์ 1 มิติ" (Vector) เพื่อเตรียมป้อนเข้าสู่สมองส่วนตัดสินใจ
6	Dense	128 neurons, ReLU	Classification: เปรียบเสมือนสมองส่วนประมวลผลความสัมพันธ์ นำฟีเจอร์ทั้งหมดที่สกัดได้มาประมวลผลร่วมกันเพื่อหาความเชื่อมโยง

7	Dropout	Rate: 0.5 (50%)	Regularization: เทคนิคสำคัญในการแก้ปัญหา Overfitting โดยการ "สุ่มปิด" การทำงานของ Neuron ครึ่งหนึ่งในทุกรอบการเทรน เพื่อบังคับให้โมเดลไม่ใช่วิธีท่องจำ (Memorization) แต่ต้องเรียนรู้ฟีเจอร์ที่กระจายตัว
8	Dense (Output)	10 neurons, Softmax	Decision Making: แปลงค่าผลลัพธ์เป็น "ความน่าจะเป็น" ของตัวเลข 0-9 ทั้ง 10 ตัว โดยผลรวมความน่าจะเป็นทั้งหมดจะเท่ากับ 1.0 (100%)

3.4. การวัดผลและเทคนิคการเทรน (Training & Evaluation)

3.4.1. Loss Function: ใช้ Categorical Crossentropy ซึ่งเป็นมาตรฐานสำหรับโจทย์จำแนกประเภทที่มีหลายคำตอบ (Multi-class) สมการทางคณิตศาสตร์คือ:

$$Loss = - \sum_{i=1}^C y_i \cdot \log(\hat{y}_i)$$

โดยที่ y_i คือค่าจริง (One-hot) และ $\hat{y}_i$ คือค่าที่โมเดลทำนาย ยิ่งโมเดลทำนายผิด ค่า Loss จะยิ่งสูงมาก เป็นการลงโทษ (Penalize) โมเดลให้ปรับตัว

3.4.2. Optimizer: ใช้ Adam (Adaptive Moment Estimation) ซึ่งเป็นอัลกอริทึมการหาค่าต่ำสุดที่ฉลาดกว่า Gradient Descent ธรรมดา โดยสามารถปรับ Learning Rate ให้เหมาะสมกับพารามิเตอร์แต่ละตัวได้เอง ทำให้การเทรนรวดเร็วและมีประสิทธิภาพ

3.4.3. Data Augmentation: ใน EP 15 มีการใช้เทคนิค ImageDataGenerator เพื่อสร้างข้อมูลเทียมเพิ่ม โดยการ หมุนภาพ (Rotation), เลื่อนภาพ (Shift), และซูมภาพ (Zoom) เล็กน้อย เพื่อจำลองความหลากหลายของลายมือมนุษย์ที่อาจเขียนเอียงหรือเขียนไม่ตรงกึ่งกลาง ช่วยให้โมเดลเก่งขึ้น (Generalization) และทนทานต่อข้อมูลใหม่ๆ

4. กรณีศึกษาที่ 2: ระบบจดจำใบหน้าและการถ่ายโอนการเรียนรู้ (Face Recognition & Transfer Learning)

เมื่อโจทย์มีความซับซ้อนขึ้นถึงระดับ "การระบุตัวตนบุคคลจากใบหน้า" (Face Recognition) การสร้างโมเดล CNN จากศูนย์ (Training from Scratch) แบบเดียวกับ MNIST จะไม่สามารถทำได้ง่ายๆ เนื่องจากใบหน้ามนุษย์มีความซับซ้อนสูงและต้องใช้ข้อมูลจำนวนมากในการสอน บทนี้จึงนำเสนอแนวคิด Transfer Learning หรือการ "ยืมบ่นไหลดักซ์"

4.1. จาก Classification สู่ Metric Learning

ใน MNIST เป้าหมายคือการตอบว่าภาพนี้คือเลขอะไร (Classification) แต่ในงานจดจำใบหน้า โจทย์คือการเปรียบเทียบว่า "หน้า A เหมือนกับหน้า B หรือไม่?" ซึ่งเป็นปัญหาแบบ Open-set (จำนวนคนในโลกมีไม่จำกัด ไม่สามารถกำหนด Class ตายตัว 10 ตัวเหมือนตัวเลขได้)

ดังนั้น เราจึงเปลี่ยนกระบวนทัศน์มาใช้ Metric Learning โดยเป้าหมายไม่ใช่การจำแนกกลุ่ม แต่เป็นการแปลงภาพใบหน้าให้เป็น Embedding Vector (รหัสตัวเลข) ที่มีความยาวคงที่ (เช่น 128 หรือ 512 ตัวเลข)

กฎสำคัญ: ใบหน้าของคนคนเดียวกัน ไม่ว่าจะถ่ายมุมไหน แสงแบบใด เมื่อแปลงเป็น Embedding แล้ว ค่าตัวเลขทางคณิตศาสตร์ต้อง "อยู่ใกล้กัน" (Distance น้อย) ส่วนใบหน้าคนละคนต้อง "อยู่ไกลกัน"

4.2. สถาปัตยกรรมระบบจดจำใบหน้า (Face Recognition Pipeline)

ระบบที่สมบูรณ์ประกอบด้วย 2 โมเดลทำงานประสานกัน (Cascaded System):

4.2.1. การตรวจจับใบหน้าที่ด้วย MTCNN (Multi-task Cascaded Convolutional Networks)

ก่อนที่จะจดจำได้ว่า "ใคร" ต้องรู้ก่อนว่า "หน้าอยู่ที่ไหน" อัลกอริทึม MTCNN ทำหน้าที่นี้อย่างมีประสิทธิภาพสูงสุดด้วยโครงสร้าง "ตัวกรอง 3 ชั้น":

4.2.1.1. Stage 1: P-Net (Proposal Network): สแกนภาพแบบหยาบๆ อย่างรวดเร็ว เพื่อหาพื้นที่ที่ "น่าจะเป็นใบหน้า" และสร้างกรอบสี่เหลี่ยม (Bounding Box) จำนวนมาก

4.2.1.2. Stage 2: R-Net (Refinement Network): รับผลจาก P-Net มากรองซ้ำ ตัดกล่องที่ความมั่นใจต่ำทิ้งไป (Reject False Positives) และปรับตำแหน่งกล่องให้แม่นยำขึ้น

4.2.1.3. Stage 3: O-Net (Output Network): ขั้นตอนสุดท้ายที่ละเอียดที่สุด นอกจากจะระบุตำแหน่งใบหน้าแล้ว ยังทำหน้าที่ระบุ Landmarks 5 จุด (ตาซ้าย, ตาขวา, จมูก, มุมปากซ้าย, มุมปากขวา) ซึ่งสำคัญมากสำหรับการทำ Face Alignment (หมุนภาพให้หน้าตรง) ก่อนส่งไปวิเคราะห์ต่อ

4.2.2. การระบุตัวตนด้วย InceptionResnetV1 (Feature Extractor)

เมื่อได้ภาพใบหน้าที่ตัดเฉพาะส่วน (Cropped Face) ขนาดมาตรฐาน 160×160 พิกเซลแล้ว ข้อมูลจะถูกส่งเข้าสู่โมเดลหลักคือ InceptionResnetV1 ซึ่งมีคุณสมบัติพิเศษจากเทคนิค Transfer Learning:

4.2.2.1. Pretrained Weights: โมเดลนี้ไม่ได้เริ่มเรียนรู้จากศูนย์ แต่ถูกฝึกสอน (Pretrained) มาแล้วด้วยชุดข้อมูล VGGFace2 ซึ่งประกอบด้วยภาพใบหน้าคนกว่า 3.3 ล้านภาพ ทำให้มันรู้จักโครงสร้างใบหน้านมนุษย์เป็นอย่างดี

4.2.2.2. Architecture: เป็นลูกผสมระหว่างสถาปัตยกรรม Inception (ที่ใช้ Kernel หลายขนาด $1 \times 1, 3 \times 3, 5 \times 5$ ทำงานขนานกันเพื่อจับรายละเอียดทุกมิติ) และ ResNet (ที่มี Residual Connection หรือทางด่วนข้อมูล เพื่อแก้ปัญหา Gradient หายในโมเดลที่ลึกมากๆ)

4.3. การวัดความเหมือนด้วย Cosine Similarity

เมื่อ InceptionResnetV1 แปลงภาพใบหน้าเป็น Embedding Vector แล้ว ขั้นตอนสุดท้ายคือการเปรียบเทียบความเหมือน เราเลือกใช้ Cosine Similarity ซึ่งวัด "มุม" ระหว่างเวกเตอร์สองตัว แทนที่จะวัดระยะห่างทางกายภาพ (Euclidean Distance)

สูตรทางคณิตศาสตร์:

$$\text{CosineSimilarity} = \frac{A \cdot B}{|A||B|} = \frac{\sum A_i B_i}{\sqrt{\sum A_i^2} \sqrt{\sum B_i^2}}$$

4.3.1. ผลลัพธ์: ค่าจะอยู่ระหว่าง -1 ถึง 1

4.3.1.1. เข้าใกล้ 1.0: เวกเตอร์ชี้ไปทางเดียวกัน (มุม 0 องศา) = ใบน้ําเดียวกัน (Match)

4.3.1.2. เข้าใกล้ 0 หรือติดลบ: เวกเตอร์ตั้งฉากหรือชี้สวนทาง = คนละคน (Mismatch)

4.3.2. Thresholding: ระบบจะกําหนดค่าเกณฑ์ (เช่น 0.7) หากค่า Similarity สูงกว่าเกณฑ์นี้ ก็จะสรุปว่าเป็นบุคคลเดียวกัน การตั้งค่า Threshold นี้เป็นศิลปะที่ต้องปรับจูน (Fine-tuning) เพื่อสมดุลระหว่างความปลอดภัย (Security) และความสะดวก (Convenience)

5. บทสรุป การประยุกต์ใช้ และจริยธรรม (Conclusion, Applications, and Ethics)

5.1. การสังเคราะห์องค์ความรู้

ใบความรู้นี้ได้นําพาท่านเดินทางผ่านวิวัฒนาการทางเทคโนโลยี ตั้งแต่ชีววิทยาของเซลล์ประสาทสู่สมการคณิตศาสตร์ (Perceptron) การปฏิวัติวงการด้วย ReLU และ CNN จนถึงการประยุกต์ใช้ขั้นสูงในระบบ Face Recognition เราได้เห็นแล้วว่า หัวใจสำคัญของ AI ไม่ใช่เวทมนตร์ แต่คือ คณิตศาสตร์ของการหาค่าที่เหมาะสม (Optimization) และ สถาปัตยกรรมที่เลียนแบบธรรมชาติ (Biomimetic Architecture)

5.1.1. MNIST สอนให้เราเข้าใจกระบวนการพื้นฐาน: Input → Feature Extraction → Classification

5.1.2. Face Recognition สอนให้เราเข้าใจการต่อยอด: การใช้ Pretrained Model และการเปลี่ยนวิธีคิดจากการจำแนก Class เป็นการวัดระยะห่าง (Metric Learning)

5.2. ความท้าทายและจริยธรรม (Ethics & Challenges)

ในฐานะผู้พัฒนาและผู้ใช้งาน AI เราต้องตระหนักถึงประเด็นที่เหนือกว่าเทคนิค :

5.2.1. อคติของข้อมูล (Data Bias): หากโมเดล (เช่น InceptionResnetV1) ถูกเทรนด้วยภาพคนผิวขาวเป็นส่วนใหญ่ มันอาจทำงานผิดพลาดกับคนผิวสีหรือชนชาติอื่น ซึ่งนำไปสู่การเลือกปฏิบัติทางเชื้อชาติ (Racial Discrimination) ในระบบรักษาความปลอดภัย

5.2.2. ความเป็นส่วนตัว (Privacy): ข้อมูลใบหน้าเป็นข้อมูลชีวมาตร (Biometric Data) ที่มีความละเอียดอ่อน การจัดเก็บและประมวลผลต้องเป็นไปตามกฎหมายคุ้มครองข้อมูลส่วนบุคคล (เช่น PDPA ในไทย) อย่างเคร่งครัด

5.2.3. ความปลอดภัย (Security): ระบบ Face Recognition อาจถูกหลอกได้ด้วยรูปภาพหรือหน้ากาก (Spoofing) การพัฒนาระบบ Liveness Detection จึงเป็นสิ่งจำเป็นสำหรับแอปพลิเคชันที่มีความเสี่ยงสูง

อ้างอิง

Aitude. (n.d.). *Comparison of Sigmoid, Tanh and ReLU activation functions*.

<https://www.aitude.com/comparison-of-sigmoid-tanh-and-relu-activation-functions/>

Analytics Vidhya. (n.d.). *In and out of FaceNet*. Medium. [https://medium.com/analytics-](https://medium.com/analytics-vidhya/in-and-out-of-facenet-41822ff6a87)

[vidhya/in-and-out-of-facenet-41822ff6a87](https://medium.com/analytics-vidhya/in-and-out-of-facenet-41822ff6a87)

Bdhuma. (n.d.). *Which pooling method is better: Maxpooling vs minpooling vs average*

pooling. Medium. [https://medium.com/@bdhuma/which-pooling-method-is-better-](https://medium.com/@bdhuma/which-pooling-method-is-better-maxpooling-vs-minpooling-vs-average-pooling-95fb03f45a9)

[maxpooling-vs-minpooling-vs-average-pooling-95fb03f45a9](https://medium.com/@bdhuma/which-pooling-method-is-better-maxpooling-vs-minpooling-vs-average-pooling-95fb03f45a9)

Brownlee, J. (n.d.). *A gentle introduction to the rectified linear unit (ReLU)*. Machine Learning

Mastery. [https://machinelearningmastery.com/rectified-linear-activation-function-for-](https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/)

[deep-learning-neural-networks/](https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/)

Brownlee, J. (n.d.). *Padding and stride for convolutional neural networks*. Machine Learning

Mastery. [https://machinelearningmastery.com/padding-and-stride-for-convolutional-](https://machinelearningmastery.com/padding-and-stride-for-convolutional-neural-networks/)

[neural-networks/](https://machinelearningmastery.com/padding-and-stride-for-convolutional-neural-networks/)

DataCamp. (n.d.). *Introduction to Convolutional Neural Networks (CNNs)*.

[https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-](https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns)

[cnns](https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns)

Deeplizard. (n.d.). *Max pooling*. https://deeplizard.com/learn/video/ZjM_XQa5s6s

Fonseca, L. (n.d.). *A practical comparison between CNN and ResNet architectures: A focus*

on attention mechanisms. Medium. [https://medium.com/@leonardofonseca.r/a-](https://medium.com/@leonardofonseca.r/a-practical-comparison-between-cnn-and-resnet-architectures-a-focus-on-attention-mechanisms-cee7ec8eca55)

[practical-comparison-between-cnn-and-resnet-architectures-a-focus-on-attention-](https://medium.com/@leonardofonseca.r/a-practical-comparison-between-cnn-and-resnet-architectures-a-focus-on-attention-mechanisms-cee7ec8eca55)

[mechanisms-cee7ec8eca55](https://medium.com/@leonardofonseca.r/a-practical-comparison-between-cnn-and-resnet-architectures-a-focus-on-attention-mechanisms-cee7ec8eca55)

GeeksforGeeks. (n.d.). *CNN | Introduction to pooling layer*.

<https://www.geeksforgeeks.org/deep-learning/cnn-introduction-to-pooling-layer/>

GeeksforGeeks. (n.d.). *Derivative of the Softmax function and the categorical cross-entropy*

loss. [https://www.geeksforgeeks.org/machine-learning/derivative-of-the-softmax-](https://www.geeksforgeeks.org/machine-learning/derivative-of-the-softmax-function-and-the-categorical-cross-entropy-loss/)

[function-and-the-categorical-cross-entropy-loss/](https://www.geeksforgeeks.org/machine-learning/derivative-of-the-softmax-function-and-the-categorical-cross-entropy-loss/)

GeeksforGeeks. (n.d.). *Introduction to convolution neural network*.

<https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/>

GeeksforGeeks. (n.d.). *ML / Inception network v1*. <https://www.geeksforgeeks.org/machine-learning/ml-inception-network-v1/>

GeeksforGeeks. (n.d.). *ReLU activation function in deep learning*.

<https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/>

Lim, D. (n.d.). *The difference between artificial and biological neurons is beyond comparison*. Medium. <https://medium.com/@don-lim/the-difference-between-artificial-and-biological-neurons-is-beyond-comparison-35d85d1f8627>

MDPI. (2025). *Technologies*, 13(10), 450. <https://www.mdpi.com/2227-7080/13/10/450>

MNIST database. (2024, May 20). In *Wikipedia*. https://en.wikipedia.org/wiki/MNIST_database

MTCNN. (n.d.). *Stages*. ReadTheDocs. <https://mtcnn.readthedocs.io/en/latest/stages/>

National Institutes of Health. (n.d.). *Article PMC8320316*. PubMed Central.

<https://pmc.ncbi.nlm.nih.gov/articles/PMC8320316/>

Poloclub. (n.d.). *CNN explainer*. <https://poloclub.github.io/cnn-explainer/>

ResearchGate. (n.d.). *A simplified architecture of Inception-Resnet-v1 network* [Figure].

https://www.researchgate.net/figure/A-simplified-architecture-of-Inception-Resnet-v1-network-The-Stem-is-a-particular_fig2_324850578

Sophos. (n.d.). *Man vs. machine: Comparing artificial and biological neural networks*.

<https://www.sophos.com/en-us/blog/man-vs-machine-comparing-artificial-and-biological-neural-networks>

Sourour, O. (n.d.). *Max pooling in CNNs: Why it matters and how it works*. Medium.

<https://medium.com/@onally.sourour9/max-pooling-in-cnns-why-it-matters-and-how-it-works-60c590bca8b9>

Stack Exchange (Data Science). (2018). *When to use cosine similarity over Euclidean similarity?* [Online Q&A].

<https://datascience.stackexchange.com/questions/27726/when-to-use-cosine-similarity-over-euclidean-similarity>

Svitla Systems. (n.d.). *Math at the heart of CNN*. <https://svitla.com/blog/math-at-the-heart-of-cnn/>

The Modern Scientist. (n.d.). *Multi-task cascaded convolutional neural network (MTCNN)*. Medium. <https://medium.com/the-modern-scientist/multi-task-cascaded-convolutional-neural-network-mtcnn-a31d88f501c8>

U.S. Commission on Civil Rights. (2024, September). *Civil rights implications of facial recognition technology*. https://www.usccr.gov/files/2024-09/civil-rights-implications-of-frt_0.pdf

Ultralytics. (n.d.). *MNIST dataset*. <https://docs.ultralytics.com/datasets/classify/mnist/>

V7 Labs. (n.d.). *Cross-entropy loss guide*. <https://www.v7labs.com/blog/cross-entropy-loss-guide>

V7 Labs. (n.d.). *Neural networks activation functions*. <https://www.v7labs.com/blog/neural-networks-activation-functions>